Why is hexadecimal used in computer science?

Programmers often use hexadecimal when writing and analysing computer programs. They do this because it is easy for a human to read hexadecimal, rather than binary.

For example, if a programmer looked at a set of data that is stored in registers, as binary it could look like this:

10011100101110111000111011100010111101011010

The programmer may be trying to find out if the data stored matches the correct data which should be:

10011100101110111001011011100010111101011010

At a glance, it is difficult to see whether the two values match or if an error has occurred.

If that stored data is represented in hexadecimal it would be:

9CBB8EE2F5A

Comparing it to the correct value that should be:

9CBB96E2F5A

It is much easier to see that the data does not match, and the programmer can identify that an error has occurred.

The benefits of using hexadecimal rather than binary for a programmer are:

- Hexadecimal is a shorter way of representing binary, so it takes up less space on the computer screen or piece of paper.
- Hexadecimal is easier for a programmer to read, write and understand.
- Hexadecimal is easier to debug and locate errors in data.

ACTIVITY 1.1

Each computer has a **MAC address**. This MAC address is written in hexadecimal. This is one of the ways that hexadecimal is used in computer science.

Use the internet to find three more uses of hexadecimal in computer science.

KEY WORD

MAC address: a unique address that is given to a computer at the manufacturing stage that can be used to identify the computer.

1.2 Binary manipulation and negative numbers

Binary addition

You need to be able to add together two 8-bit binary numbers. When binary numbers are added together there are four rules that you will need to use:

Rule 1 is 0 + 0 = 0

Rule 2 is 1 + 0 = 1

Rule 3 is 1 + 1 = 10 (this is because 10 in binary is 2)

Rule 4 is 1 + 1 + 1 = 11 (this is because 11 in binary is 3)

You can apply these rules to add the binary numbers 10010100 and 00011110.

	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0

You need to start at the right and begin applying the rules. The first calculation is 0 + 0. Rule

The next calculation is 0 + 1. Rule 2 means the result is 1.

	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
							1	**0**

The next calculation is 1 + 1. Rule 3 means the result is 10. You need to write the 0 in the answer section and carry the 1 to the next calculation.

					1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
						0	**1**	**0**

The next calculation is 1 + 0 + 1, which results in 1 + 1. Rule 3 means the result is 10. Write the 0 in the answer section again and carry the 1.

				1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
					0	**0**	**1**	**0**

The next calculation is 1 + 1 + 1. Rule 4 means the result is 11. Write 1 in the answer section and carry 1.

			1	1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
				1	**0**	**0**	**1**	**0**

The next calculation is 1 + 0 + 0, which results in 1 + 0. Rule 2 means the result is 1. Write 1 in the answer section. There is no carry this time.

			1	1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
			1	**1**	**0**	**0**	**1**	**0**

The next calculation is 0 + 0. Rule 1 means the result is 0. Write 0 in the answer section.

			1	1	1			
+	1	0	0	1	0	1	0	0
	0	0	0	1	1	1	1	0
		0	**1**	**1**	**0**	**0**	**1**	**0**

The final calculation is 1 + 0. Rules 2 means the result is 1. Write 1 in the answer section.

			1	1	1			
+	1	0	0	1	0	1	0	0
	0	0	0	1	1	1	1	0
	1	**0**	**1**	**1**	**0**	**0**	**1**	**0**

Therefore, when the binary numbers 10010100 and 00011110 are added, the answer is 10110010.

Questions

3 Add the 8-bit binary values 00011001 and 01000011.

4 Add the 8-bit binary values 00100110 and 01010111.

You should now know that the largest number that can be stored in an 8-bit binary register is 255. What happens when two 8-bit binary values are added, and the result is larger than 255?

The following example, which is almost complete, is very much like the previous calculation with a slightly different binary number. Can you spot the difference?

			1	1	1			
+	1	0	0	1	0	1	0	0
	1	0	0	1	1	1	1	0
		0	**1**	**1**	**0**	**0**	**1**	**0**

If you now try and do the final calculation, that would be 1 + 1 = 10. You write 0 in the answer section, but there is nowhere to carry the 1. This is because the two 8-binary binary values are added the result is 306, which is greater than 255. Therefore, the number cannot fit in the 8-bit register and an **overflow error** is created.

			1	1	1			
+	1	0	0	1	0	1	0	0
	1	0	0	1	1	1	1	0
(1)	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**0**

Overflow error

Each computer has a predefined limit, for example, 16-bit registers. What is the largest number that could be stored in a 16-bit register? If a number larger than this is generated in a process, an overflow error will occur.

KEY WORD

overflow error: a type of error that occurs when a number larger than a register can store is generated.

Logical binary shifts

Another process than can occur with binary numbers that are stored in a register is called a logical binary shift. This kind of shift can move the binary values in the number to the left or to the right.

Consider the binary number:

0	0	1	0	1	1	0	0

If a logical binary shift to the left is performed on the binary number, each value in the binary number is moved one place to the left. The result would be:

0	1	0	1	1	0	0	

You should notice that the left most 0 has been removed from the register. This is because there was no place to the left to shift this value. You should also notice that there is now an empty section in the register at the right. When a binary number is shifted, the resulting empty sections are filled with a 0.

0	1	0	1	1	0	0	0

If you convert the first binary number to denary, you get the denary number 44. If you convert the binary number after the logical left shift has been performed, you get the denary number 88. This means that each logical shift to the left multiples the binary number by 2.

Consider the binary number again:

0	0	1	0	1	1	0	0

If a logical shift to the right is performed on the binary number, each value in the binary number is moved one place to the right. The result would be:

	0	0	1	0	1	1	0

You should notice that the right most 0 has been removed from the register. This is because there was no place to the right to shift this value. You should also notice that there is now an empty section in the register at the left. When a binary number is shifted, the resulting empty sections are, again, filled with a 0.

0	0	0	1	0	1	1	0

You already know that the first binary number converted to a denary number is 44. If you convert the binary number after the logical right shift has been performed, you get the denary number 22. This means that each logical shift to the right divides the binary number by 2.

Consider another binary number:

1	1	1	0	1	1	0	0

When a logical left shift is performed on the binary value the result is:

1	1	0	1	1	0	0	0

When the first binary number is converted to a denary number, the result is 236. When the binary number is converted after the logical shift to the left has been performed, the denary value is 216. You should notice that 236 multiplied by 2 is not 216. Therefore, one of the problems with a logical shift is that data can lose its integrity as vital bits can be removed from the register.

More than one logical shift can be performed on a register. For example, two logical shifts to the left can be performed. This time, each value in the register is shifted two places to the left.

The binary value:

1	1	1	0	1	1	0	0

would become:

1	0	1	1	0	0	0	0

Questions

5 Describe what happens in a logical binary shift.

6 Perform **two** logical shifts to the right on the binary number 11101100.

Negative binary numbers

All the binary numbers that you have used so far have been positive binary numbers. Numbers can sometimes be negatives number though, for example, −150. How are negative numbers represented as binary? Binary only has two values that can be used, a 0 and a 1, so you can't just put a minus symbol (−) in front of the number, as that symbol doesn't exist in binary. Therefore, a method exists that can be used to represent negative binary numbers and it is called two's complement. Most modern computers use this method.

The method of two's complement is a simple one. First you invert all the values in the binary number (change the 1s to 0s and the 0s to 1s), then you add 1 to the result. You can use this process to find out how the denary value −35 is represented as binary.

Convert 35 to a binary number:

0	0	1	0	0	0	1	1

Invert each of the values in the binary number:

1	1	0	1	1	1	0	0

Then simply add 1:

1	1	0	1	1	1	0	1

11011101 is the binary representation of −35 using the two's complement method.